

SECURE LOGIN SYSTEM

Objective

The objective of this project is to develop a secure web-based login system that protects user accounts from unauthorized access. The application allows users to register and log in using encrypted passwords. It also includes input validation, protection against SQL injection attacks, session management, logout functionality, and an optional Two-Factor Authentication (2FA) feature to enhance security.

Problem Statement

Many traditional login systems store passwords in plain text, making them vulnerable to data breaches and cyber attacks. They may also be susceptible to SQL injection attacks and unauthorized access due to poor session management. This project addresses these security concerns by implementing password hashing using BCrypt, parameterized SQL queries to prevent SQL injection, proper session handling, and secure user authentication.

Introduction

A secure login system is an essential part of modern web applications. It authenticates users before granting access to protected resources. This project demonstrates the implementation of industry-standard security practices including password encryption, session management, and secure database operations.

The system ensures that user passwords are never stored in plain text. Instead, passwords are encrypted using the BCrypt hashing algorithm before being saved in the database. The application also validates user inputs and uses parameterized SQL queries to eliminate SQL injection vulnerabilities.

Software Requirements

- Python 3.10 or Above
- Flask
- Flask-Bcrypt
- SQLite3
- HTML
- CSS
- Visual Studio Code (Recommended)

Hardware Requirements

- Processor: Intel i3 or Above
 - RAM: 4 GB Minimum
 - Storage: 100 MB Free Space
 - Operating System: Windows/Linux/macOS
-

Python Libraries Used

Flask

Flask-Bcrypt

SQLite3

os

Install required libraries:

```
pip install flask flask-bcrypt
```

Technologies Used

- Python
 - Flask Framework
 - SQLite Database
 - BCrypt Password Hashing
 - HTML
 - CSS
-

Features

- User Registration
- Secure User Login
- Password Hashing using BCrypt
- Input Validation
- SQL Injection Protection
- Session Management
- Logout Functionality
- Optional Two-Factor Authentication (2FA)

- Secure Database Storage
-

Modules

Module 1: User Registration

Users create a new account by entering a username and password.

The system:

- Validates the input
 - Encrypts the password using BCrypt
 - Stores the username and hashed password in the database
-

Module 2: User Login

Users log in using their registered credentials.

The application:

- Searches for the username
 - Compares the entered password with the hashed password
 - Grants access if the credentials are correct
-

Module 3: Password Hashing

Passwords are encrypted using BCrypt before storage.

Example

Original Password

Hello@123

Stored Password

\$2b\$12\$6fE4M4.....

The original password is never stored in the database.

Module 4: Input Validation

The application validates user input before processing.

Checks include:

- Username cannot be empty

- Password cannot be empty
 - Password should contain at least 8 characters
 - Invalid characters are rejected
-

Module 5: SQL Injection Protection

Parameterized SQL queries are used.

Unsafe Query

```
SELECT * FROM users WHERE username='admin'
```

Safe Query

```
cursor.execute(  
"SELECT * FROM users WHERE username=?",  
(username,))
```

This prevents SQL Injection attacks.

Module 6: Session Management

After successful login,

A secure session is created.

Protected pages are accessible only after authentication.

The session is destroyed after logout.

Module 7: Logout

Users can securely end their session by clicking the Logout button.

This removes all session information and redirects the user to the login page.

Module 8: Optional Two-Factor Authentication (2FA)

An OTP can be generated after successful password verification.

Working Principle

Step 1

User opens the registration page.

↓

Step 2

User enters username and password.



Step 3

Password is encrypted using BCrypt.



Step 4

User details are stored in SQLite database.



Step 5

User opens the login page.



Step 6

User enters username and password.



Step 7

Entered password is compared with the stored hashed password.



Step 8

If correct,

Session starts.



Step 9

User accesses the dashboard.



Step 10

User logs out.



Session ends.

Python Source Code

Step 1: Install Libraries

Open Command Prompt and run:

```
pip install flask flask-bcrypt
```

Step 2: Create Folder Structure

SecureLoginSystem/

```
| app.py
|
| users.db (Automatically Created)
|
|
| — templates/
|
|   register.html
|
|   login.html
|
|   dashboard.html
```

Step 3: app.py

```
from flask import Flask, render_template, request, redirect, session, flash
from flask_bcrypt import Bcrypt
import sqlite3
app = Flask(__name__)
app.secret_key = "secretkey123"
bcrypt = Bcrypt(app)

# Create Database
def init_db():
    conn = sqlite3.connect("users.db")
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS users( id INTEGER PRIMARY KEY
        AUTOINCREMENT, username TEXT UNIQUE, password TEXT)
    """)
    conn.commit()
    conn.close()
```

```

init_db()
@app.route("/")
def home():
    return redirect("/login")

# Registration
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form["username"].strip()
        password = request.form["password"]
        if username == "" or password == "":
            flash("Fields cannot be empty")
            return redirect("/register")
        if len(password) < 8:
            flash("Password must be at least 8 characters")
            return redirect("/register")
        hashed = bcrypt.generate_password_hash(password).decode("utf-8")
        try:
            conn = sqlite3.connect("users.db")
            cur = conn.cursor()
            cur.execute(
                "INSERT INTO users(username,password) VALUES(?,?)",
                (username, hashed)
            )
            conn.commit()
            conn.close()
            flash("Registration Successful")
            return redirect("/login")
        except:
            flash("Username already exists")
            return redirect("/register")

    return render_template("register.html")

# Login
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]
        conn = sqlite3.connect("users.db")
        cur = conn.cursor()
        cur.execute(
            "SELECT * FROM users WHERE username=?",
            (username,)

```

```

    )
    user = cur.fetchone()
    conn.close()
    if user and bcrypt.check_password_hash(user[2], password):
        session["user"] = username
        return redirect("/dashboard")
    else:
        flash("Invalid Username or Password")
    return render_template("login.html")

# Dashboard
@app.route("/dashboard")
def dashboard():
    if "user" not in session:
        return redirect("/login")
    return render_template("dashboard.html", user=session["user"])

# Logout
@app.route("/logout")
def logout():
    session.pop("user", None)
    flash("Logged Out Successfully")
    return redirect("/login")
if __name__ == "__main__":
    app.run(debug=True)

```

Step 4: templates/register.html

```

<!DOCTYPE html>
<html>
<head>
<title>Register</title>
</head>
<body>
<h2>User Registration</h2>
<form method="POST">
Username<br>
<input type="text" name="username"><br><br>
Password<br>
<input type="password" name="password"><br><br>
<input type="submit" value="Register">
</form>
<p style="color:red;">
{% with messages = get_flashed_messages() %}
{% if messages %}
{{ messages[0] }}
{% endif %}

```



```
{% endwith %}
</p>
<a href="/login">Already Registered? Login</a>
</body>
</html>
```

Step 5: templates/login.html

```
<!DOCTYPE html>
<html>
<head>
<title>Login</title>
</head>
<body>
<h2>User Login</h2>
<form method="POST">
Username<br>
<input type="text" name="username"><br><br>
Password<br>
<input type="password" name="password"><br><br>
<input type="submit" value="Login">
</form>
<p style="color:red;">
{% with messages = get_flashed_messages() %}
{% if messages %}
{{ messages[0] }}
{% endif %}
{% endwith %}

</p>
<a href="/register">Create New Account</a>
</body>
</html>
```

Step 6: templates/dashboard.html

```
<!DOCTYPE html>
<html>
<head>
<title>Dashboard</title>
</head>
<body>
<h1>Welcome {{ user }}</h1>
<h3>You have successfully logged in.</h3>
<a href="/logout">Logout</a>
</body>
</html>
```

Step 7: Run the Project

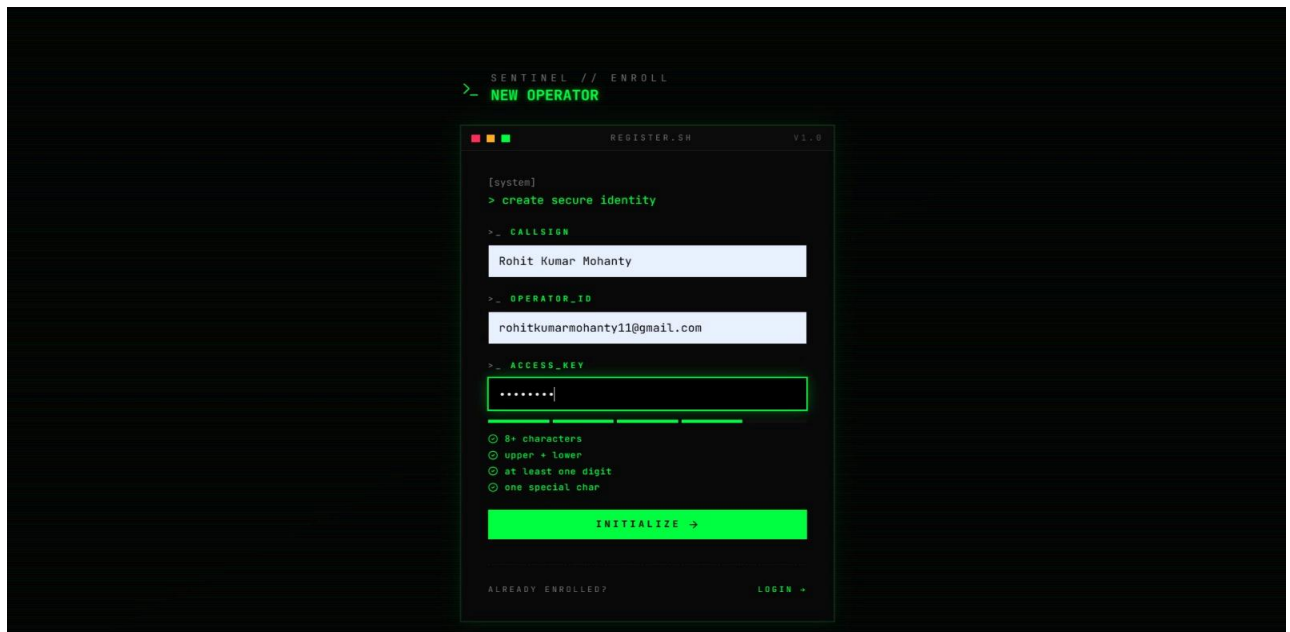
Open Terminal inside the project folder.

Run

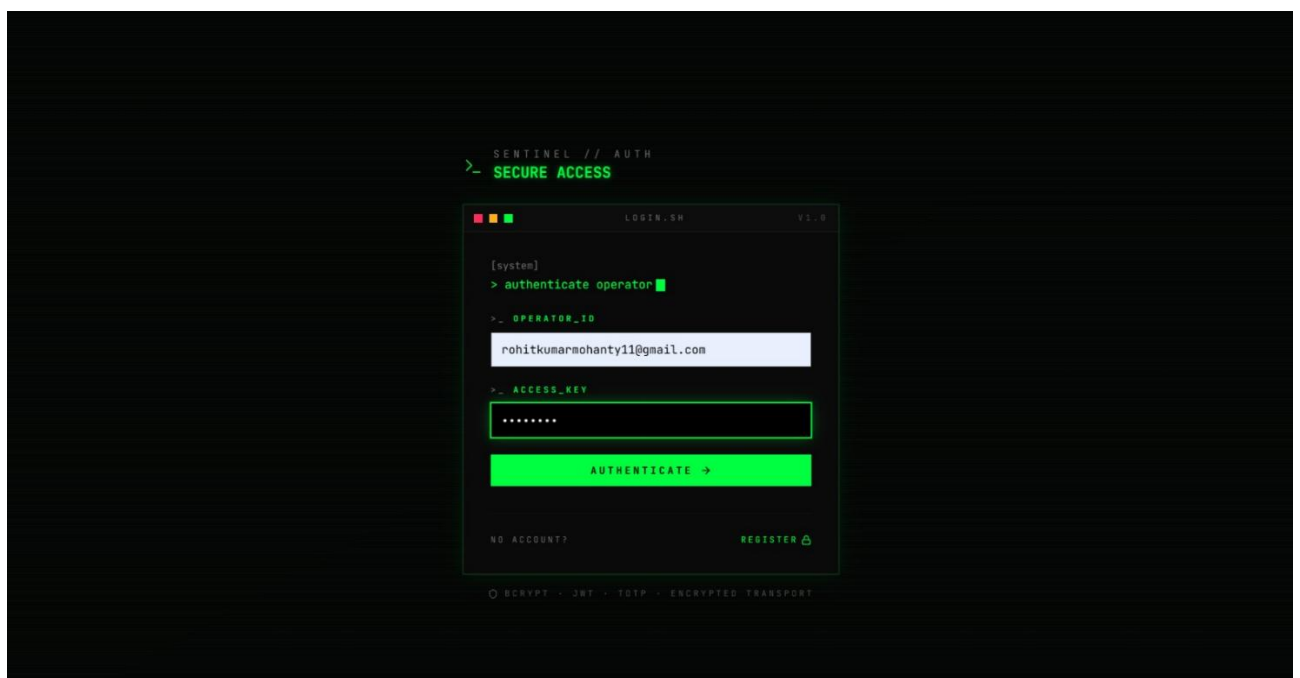
python app.py

Screenshots of the Web

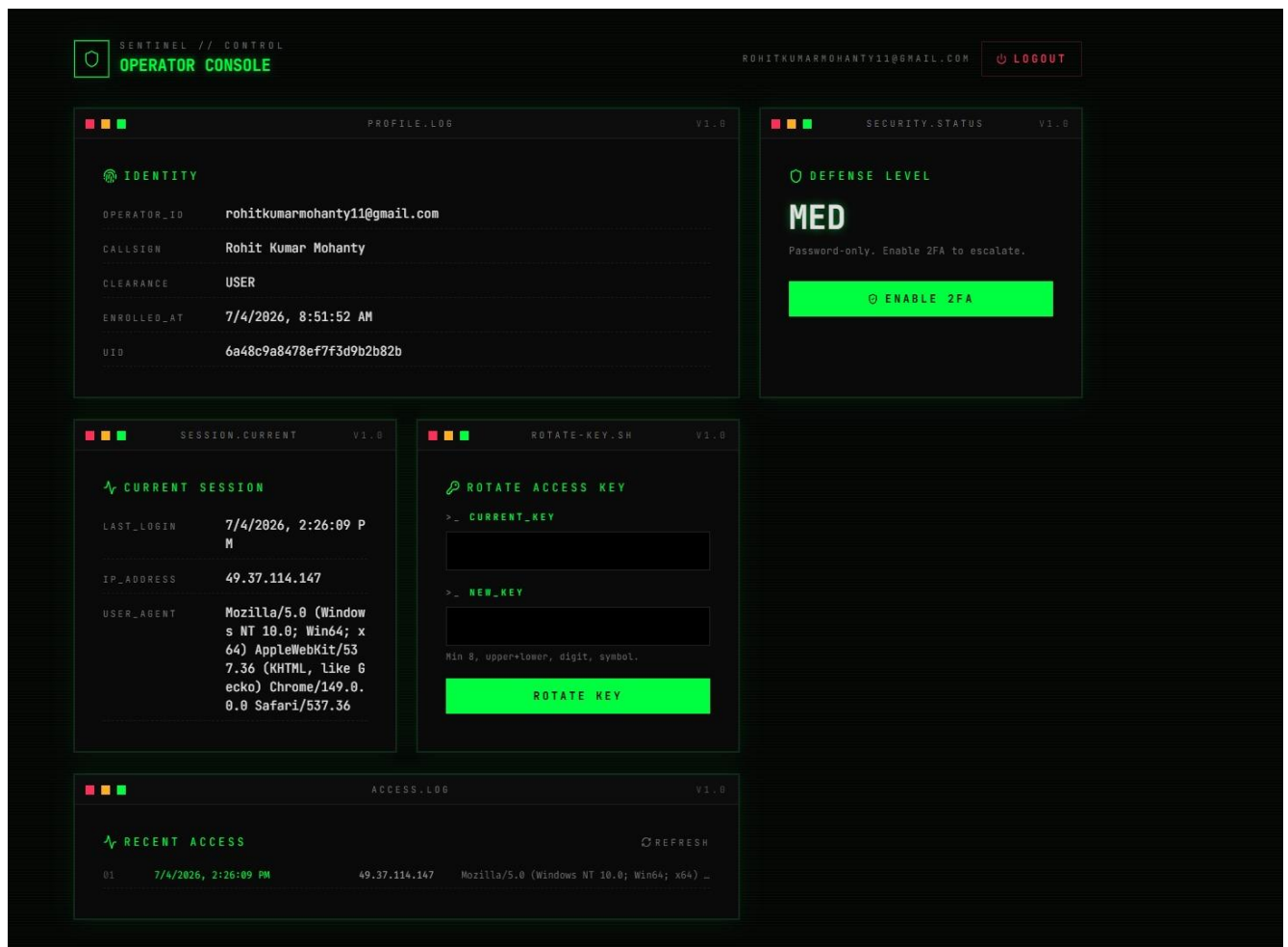
Registration:



Login:



Dashboard:



Result

The Secure Login System was successfully developed using Python, Flask, SQLite, and BCrypt. The application securely stores user passwords using hashing techniques, validates user inputs, protects against SQL Injection attacks, manages user sessions effectively, and provides a secure authentication mechanism. The system performs efficiently and meets all the required security objectives.

Conclusion

This project demonstrates the implementation of a secure authentication system using modern web security practices. Passwords are encrypted using BCrypt, user inputs are validated before processing, SQL Injection attacks are prevented using parameterized queries, and authenticated sessions are securely managed. The project is simple, efficient, and suitable for learning secure web application development while providing a strong foundation for more advanced authentication systems.